Uniwersytet Komisji Edukacji Narodowej w Krakowie

Kody Autoryzacyjne CAPTCHA

Aspekty teoretyczne i implementacja

Autor: **Igor Wróblewski**

Kraków, 2 kwietnia 2026

Spis treści

1 Streszczenie 2
2 Wykorzystanie Sztucznej Inteligencji 3
3 Historia i rozwój 4
4 Krytyka 7
5 Dlaczego Nie Warto Tworzyć Własnych CAPTCHA 9
6 Implementacja 10
7 Badanie AI 15
8 Wnioski i Refleksje 22
Glosariusz 23
Bibliografia 24

1 Streszczenie

Praca obejmuje zagadnienia począwszy od teoretycznego testu Alana Turinga – znanego jako *test Turinga* – który miał za zadanie odróżnić człowieka od maszyny, aż po rozwiązania *zero knowledge*, współcześnie wykorzystywane do weryfikacji użytkowników.
CAPTCHA, a dokładniej *Completely Automated Public Turing test to tell Computers and Humans Apart*, to automatyczny test, w którym to komputer, a nie człowiek, ocenia, czy dana odpowiedź lub zachowanie pochodzi od człowieka[1].
Historia technologii CAPTCHA to ciągły wyścig zbrojeń – projektantów zabezpieczeń z przestępcami, twórcami botów – który trwa do dziś.

W obecnym świecie, gdzie dostęp do informacji i prywatność danych często stają się wymogiem instytucjonalnym, twórcy technologii weryfikujących człowieczeństwo muszą ciągle ulepszać swój produkt tak, aby był zarówno zgodny z prawem, jak i powstrzymywał niepożądanych użytkowników.
Krytycznie należy spojrzeć na utrudnienia dla osób z niepełnosprawnościami, które do dziś mogą napotykać problemy wynikające z braku inkluzywności starszych systemów weryfikacji, wciąż niekiedy stosowanych w praktyce.
Innym istotnym problemem jest prywatność, która jest naruszana przez twórców tego typu systemów.
Inne systemy, choć bezpieczne pod względem prywatności, można sklasyfikować jako nieekologiczne – wymagają od maszyny użytkownika wykonania skomplikowanych obliczeń matematycznych.
Krótko przedstawiony zostanie problem tworzenia własnych systemów CAPTCHA i wyjaśnione zostanie, dlaczego takie rozwiązanie jest zarówno technicznie, jak i ekonomicznie całkowicie nieopłacalne.

Pochylimy się nad autorską implementacją systemu weryfikacji, w którym program, napisany w większości w języku Python, sam generuje ciągi znaków stosowane jako kody CAPTCHA.
Na podstawie tych kodów generowane są zniekształcone obrazy, które są czytelne dla człowieka, lecz nie dla maszyny.
Następnie obrazy te są wyświetlane w środowisku webowym, gdzie użytkownik przed zalogowaniem musi pomyślnie przejść taki test.
Zagłębimy się w techniczną część implementacji, gdzie przedstawione zostaną fragmenty kodu wraz z wyjaśnieniami.

Dalej przedstawione zostaną autorskie badania przeprowadzone na modelach AI.
Dzięki nim przyjrzymy się możliwościom obecnej sztucznej inteligencji oraz temu, co to oznacza dla przyszłości CAPTCHA.
Na końcu podsumowany zostanie cały temat wraz z refleksjami na temat przyszłości i samej idei systemu weryfikacji człowieczeństwa.

[1]Tak zwany odwrócony test Turinga (ang. *reverse Turing test*), w którym to maszyna jest oceniającym, a nie człowiek.

2 Wykorzystanie Sztucznej Inteligencji

Informuję, że **korzystałem ze sztucznej inteligencji** przy tworzeniu tej pracy – NotebookLM, Gemini[2].

Gemini AI zostało użyte do stworzenia, importu danych i poprawy dokumentu w programie *LaTeX*, zostało także wykorzystane jako **pomoc w pisaniu samej implementacji**, więcej w komentarzach zawartych w kodzie źródłowym.
Odpowiada również za poprawę błędów – ortograficznych, gramatycznych, stylistycznych, składniowych – w samej pracy. Sama prezentacja wyników została stworzona za pomocą *Jupyter Lab*, za którą **odpowiadało Gemini AI**.
Sytuacja jest podobna, jeśli chodzi o języki **HTML, CSS, a także moduł hashlib, bibliotekę PIL (generowanie obrazów) oraz mini-framework Flask**.

NotebookLM został użyty do researchu i ułatwienia przyswajania wiedzy; za jego pomocą zostały wyszukane artykuły i prace na temat CAPTCHA[3]. Prawie cała bibliografia to same artykuły i prace, które NotebookLM znalazł i wykorzystał.

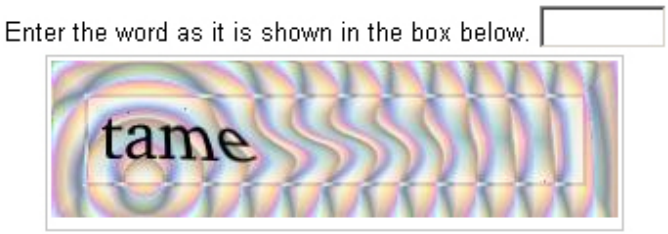
[2]Grok nie został wyliczony, jako że służył jedynie jako generator odpowiedzi do statystyk.
[3]Link do samego NotebookLM został umieszczony w *bibliografii*.

3 Historia i rozwój

Sam koncept sprawdzania człowieczeństwa zapoczątkował Alan Turing, brytyjski matematyk i kryptograf, znany ze swojego wkładu w złamanie kodu Enigmy.
Opracował on test Turinga, w którym dzięki pytaniom zadanym przez człowieka, będącego sędzią, można ocenić, czy z drugiej strony mamy do czynienia z człowiekiem, czy z maszyną.
Idea ta, choć w tamtych czasach futurystyczna, jest dziś stosowana w praktyce.
Obecnie istnieją systemy sztucznej inteligencji (AI), które przeszły test Turinga[4].
W testach weryfikacji człowieczeństwa typu CAPTCHA test Turinga również jest wykorzystywany, ale w odwróconej formie.
W tym przypadku to maszyna zadaje człowiekowi pytanie, a gdy ten odpowie poprawnie, zostaje dopuszczony do danego serwisu.
Test weryfikacyjny jest zaprojektowany w taki sposób, że maszyna nie jest w stanie na niego poprawnie odpowiedzieć.
Pierwsza implementacja systemu weryfikacji człowieczeństwa pojawiła się w 1997 roku, kiedy to wyszukiwarka internetowa AltaVista była atakowana przez boty, które utrudniały funkcjonowanie całej infrastruktury[5].
Twórcy serwisu wprowadzili test polegający na przepisaniu ciągu znaków, który dzięki zniekształceniom był niemożliwy do odczytania przez systemy OCR.

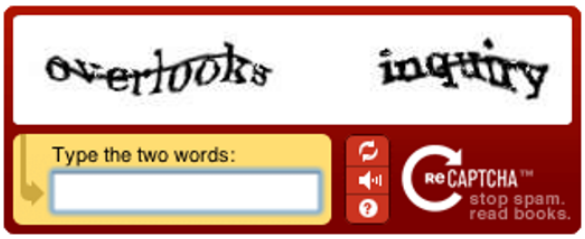
Rysunek 1: Test weryfikacyjny AltaVista[6].

W 2000 roku wyszukiwarka internetowa Yahoo! była atakowana przez boty spamujące.
Korporacja nawiązała współpracę z Carnegie Mellon University, zlecając uczelni opracowanie rozwiązania tego problemu.
Wtedy właśnie ukuto termin CAPTCHA, określający test łatwy dla człowieka, ale jednocześnie trudny dla komputera – tak zwany *AI-hard problem*.
Ostatecznie korporacja wraz z uniwersytetem opracowała rozwiązania *Gimpy* i *EZ-Gimpy*.

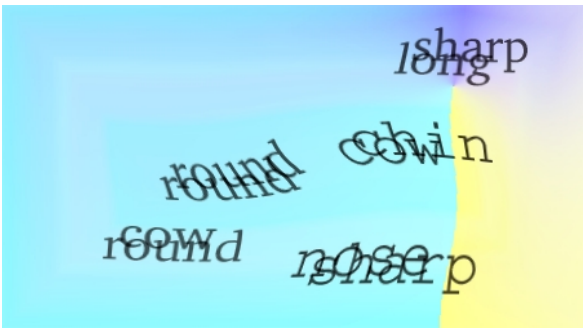


Rysunek 2: Test CAPTCHA EZ-Gimpy, podobny do testów AltaVista[7].

[4]Źródło: arxiv.org (dostęp: 7.03.2026 r.).
[5]Źródło: researchgate.net (dostęp: 7.03.2026 r.).
[6]Źródło: researchgate.net (dostęp: 6.03.2026 r.).
[7]Źródło: strona źródłowa (dostęp: 6.03.2026 r.).



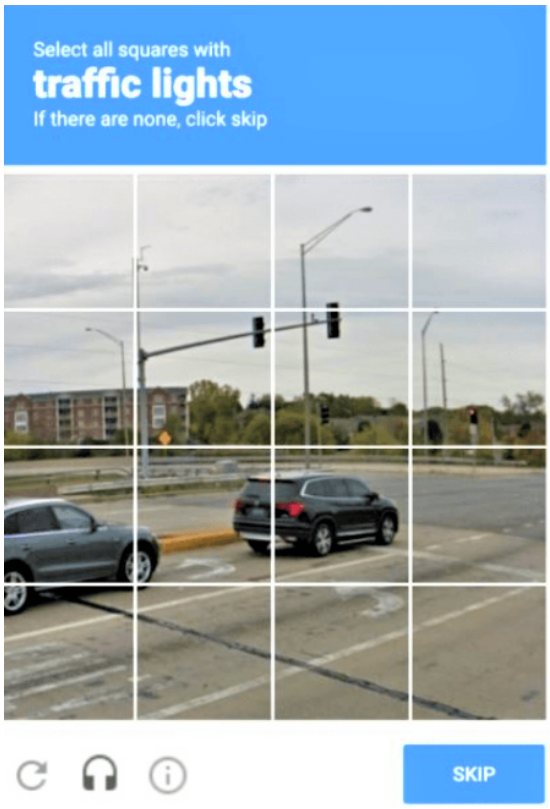
(a) ReCAPTCHA w wersji pierwszej.



(b) Gimpy – znajdź trzy takie same słowa.

Rysunek 3: Testy weryfikacji w formie tekstowej, zniekształcone, by powstrzymać boty[8].

Kolejnym systemem testów była usługa ReCAPTCHA, a rozwijająca ją firma została szybko wykupiona przez korporację Google.
Sama usługa ReCAPTCHA w wersji pierwszej była aktywna do 2018 roku, kiedy to została zastąpiona przez wersję drugą.
Już w 2014 roku korporacja poinformowała, że jej systemy AI są w stanie rozwiązać 99,8% tekstowych testów CAPTCHA. W tym momencie ReCAPTCHA ewoluowała do wersji drugiej, gdzie testy obrazkowe stały się standardem.



Rysunek 4: ReCAPTCHA w wersji drugiej, weryfikacja obrazkowa[9].

[8]Źródła: strona źródłowa (a), strona źródłowa (b) (dostęp: 6.03.2026 r.).
[9]Źródło: strona źródłowa (dostęp: 6.03.2026 r.).

W 2015 roku świat dowiedział się o praktyce firmy Google, która używała systemu ReCAPTCHA do rozpoznawania różnych obiektów na zdjęciach[10].
Korporacja została pozwana o wykorzystywanie nieodpłatnej pracy ludzkiej (rozwiązywania testów CAPTCHA) do celów biznesowych – ostatecznie jednak firma wygrała proces.
Kolejnym krokiem w rozwoju rozwiązań weryfikacji człowieczeństwa stał się system ReCAPTCHA w wersji trzeciej.
Podobnie jak w wersji 2, jest to system, który z reguły nie wymaga od użytkownika żadnej interakcji ani rozwiązywania testów.
Jeśli jednak algorytm wykryje podejrzane zachowania, użytkownik może zostać poproszony o przejście weryfikacji graficznej, choć jest to zależne od samego administratora strony[11].

Monopol ReCAPTCHA skończył się, gdy firma Google zmieniła cennik usługi weryfikacji.
Wcześniej system był darmowy dla właścicieli stron do miliona wejść miesięcznie, natomiast po zmianach limit ten drastycznie spadł do dziesięciu tysięcy.
W dodatku ReCAPTCHA w wersji trzeciej zbiera dane poufne użytkowników, naruszając ich prywatność[12].
Te dwa fakty przyczyniły się do powstania rozwiązań takich jak hCAPTCHA, które nie naruszają prywatności i oferują wyższe limity niż konkurent (technologia ta została wprowadzona przez serwis Shopify).
Należy również wspomnieć o fenomenie tzw. farm CAPTCHA.
W przypadku gdy bot nie potrafi rozwiązać danego testu (tekstowego lub obrazkowego), jest on wysyłany do farmy CAPTCHA. Tam ludzie, często pochodzący z uboższych krajów, zajmują się rozwiązywaniem tych weryfikacji w imieniu botów.

Alternatywą jest FriendlyCAPTCHA, która bazuje na dowodzie pracy (ang. *proof of work*).
Gdy użytkownik chce skorzystać ze strony chronionej tym systemem, system przydziela jego maszynie w tle zadanie matematyczne do rozwiązania.
Sam internauta nawet nie zauważa całego procesu.
Innym przykładem rozwiązania wykorzystującego protokół PoW jest Cloudflare Turnstile.
Technologia ta skutecznie zniechęca farmy botów, ponieważ masowe rozwiązywanie energochłonnych zadań matematycznych staje się dla nich nieekonomiczne.
Kolejnym rozwiązaniem, które pozornie może wydawać się kontrowersyjne z punktu widzenia prywatności, jest biometryczna CAPTCHA.
Śledzi ona ruchy myszy i dynamikę naciskania klawiszy, analizując na tej podstawie, czy zachowanie użytkownika jest naturalne dla człowieka (np. czy zawiera drobne błędy i potknięcia). System ten nie przetwarza danych wrażliwych; sprawdza jedynie, czy wzorce ruchów odpowiadają człowiekowi, czy też przypominają zautomatyzowane działanie bota.

Najnowszym trendem są tzw. dowody z wiedzą zerową (ang. *zero-knowledge proofs*), czyli systemy weryfikacji niewymagające przekazywania żadnych danych identyfikacyjnych ze strony użytkownika.
System prosi maszynę o kryptograficzne potwierdzenie tożsamości, a ona poświadcza w sposób bezpieczny, że jej użytkownikiem jest człowiek[13].

[10]Źródło: researchgate.net (dostęp: 7.03.2026 r.).
[11]Źródło: friendlycaptcha.com (dostęp: 7.03.2026 r.).
[12]ReCAPTCHA w wersji trzeciej, choć użytkowo nieuciążliwa dla internauty, w tle sprawdza historię przeglądania, pliki cookie i inne poufne dane użytkownika – system może sięgać wstecz nawet do 6 miesięcy.
[13]Źródło: researchgate.net (dostęp: 7.03.2026 r.).

4 Krytyka

Choć sam test weryfikacji człowieczeństwa w zasadzie powinien być bezproblemowy dla człowieka, obiektywne badania wskazują, że połowa użytkowników może opuścić stronę internetową po kilku nieudanych próbach weryfikacji CAPTCHA.[14]
Samo zrzucanie odpowiedzialności za bezpieczeństwo strony internetowej na użytkownika przez jej właściciela jest mechanizmem nader nieopłacalnym – strony internetowe tracą ruch.

Jednak największym problemem testów CAPTCHA jest wykluczanie osób z niepełnosprawnościami, a także toporny sposób implementowania dla nich ułatwień.
Jak wskazuje badanie z 2011 roku „CAPTCHA Accessibility Study of Online Forums", w którym zbadano 150 forów internetowych pod kątem dostępności, wynikało z niego, że średnio 61% stron stosowało testy weryfikacji tekstowe (zniekształcone znaki), które są trudne do rozszyfrowania dla osoby z niepełnosprawnością wzroku.
W tym samym czasie tylko 14% witryn oferowało alternatywę w postaci testu weryfikacji dźwiękowej[15].
Chociaż sam format dźwiękowy z pozoru zwiększa dostępność, zaledwie 46% osób z niepełnosprawnościami było w stanie go rozwiązać – podczas gdy skuteczność dobrze zaimplementowanych testów weryfikacji powinna wynosić 90%.
Testy dźwiękowe zawierają szumy i dźwięki otoczenia – by zmylić boty – które utrudniają zrozumienie ich treści.
Ponadto same testy dźwiękowe są dostępne w większości w języku angielskim, co stanowi dodatkową barierę dla kogoś nieznającego tego języka[16].
Powyższe dane jasno pokazują, że niezoptymalizowane systemy CAPTCHA budują jedynie mury, które utrudniają dostęp do internetu, w którym nawet 25% internautów to osoby z niepełnosprawnościami[17].

Rozwiązanie ReCAPTCHA, począwszy od wersji drugiej, opiera się na zbieraniu informacji o użytkowniku, jego myszy, zainstalowanych rozszerzeniach i plikach cookie.
W przypadku braku podejrzanej aktywności, po zaznaczeniu pola „nie jestem robotem", użytkownik może dalej korzystać z serwisu.
Jednakże dla osoby z niepełnosprawnością wzroku, która nie używa myszy i posługuje się klawiszem „Tab", system może być niewyrozumiały i uznać takiego użytkownika za niebezpiecznego, zmuszając go do rozwiązania graficznego testu weryfikacji[18].
Następca ReCAPTCHA 2, czyli ReCAPTCHA 3, choć jest rozwiązaniem bardziej inkluzywnym, ma jedną dużą wadę – brak poszanowania prywatności użytkownika.
Podczas gdy wersja 2 zbierała stosunkowo niewiele informacji, wersja 3 analizuje historię przeglądania i interakcje użytkownika na innych stronach w sieci, a także może wykonywać zrzuty ekranu przeglądarki – jest to technologia śledząca między witrynami.
Jeśli jednak wersja 3 uzna użytkownika za bota – w zależności od preferencji administratora – często rozwiązaniem staje się powrót do wersji 2, czyli graficznego testu weryfikacji[19].

[14]Źródło: smashingmagazine.com (dostęp 6.03.2026 r.).
[15]Źródło: researchgate.net (dostęp 9.03.2026 r.).
[16]Źródło: researchgate.net (dostęp 9.03.2026 r.).
[17]Źródło: researchgate.net (dostęp 9.03.2026 r.).
[18]Źródło: acm.org (dostęp 9.03.2026 r.).
[19]Źródło: friendlycaptcha.com (dostęp 9.03.2026 r.).

Na obecną chwilę same testy CAPTCHA w formie tekstu lub obrazku stały się kompletnie przestarzałe z perspektywy bezpieczeństwa.
Wyspecjalizowany model sztucznej inteligencji YOLO8 (ang. *you only look once*) potrafi rozwiązywać graficzne testy weryfikacji człowieczeństwa **ze 100% skutecznością**[20].
Oznacza to, że rozwiązanie ReCAPTCHA w wersji 2 i mu podobne są kompletnie bezużyteczne, a stały się jedynie dodatkowym utrudnieniem dla użytkownika, przy okazji umożliwiając zbieranie o nim danych.
Jeśli jednak chodzi o mniej wyspecjalizowane boty, które nie potrafią sprostać weryfikacji, korzystają one z usług farm CAPTCHA.
Tam człowiek rozwiązuje test CAPTCHA za bota, a następnie mu go odsyła.
Oznacza to, że bot i tak wygrywa, a prawdziwy użytkownik i tak musi taki test rozwiązać[21]. Rozwiązaniem problemu prywatności może być hCAPTCHA, która zbiera wyłącznie dane niezbędne do celów bezpieczeństwa – w przeciwieństwie do rozwiązań ReCAPTCHA, ta usługa stawia prywatność na pierwszym miejscu.
Działa na podobnych zasadach jak ReCAPTCHA w wersji 2, jednak w przypadku podejrzanego zachowania wyświetla graficzny test weryfikacji, który nie jest inkluzywny i nie zapewnia pewnego bezpieczeństwa.

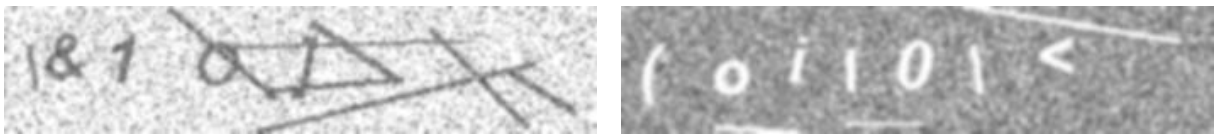
FriendlyCAPTCHA jest niewidzialnym testem człowieczeństwa – tak jak ReCAPTCHA 3.
Opiera się na mechanizmie dowodu pracy (ang. *proof of work*, PoW), w którym system przydziela maszynie użytkownika problem obliczeniowy do rozwiązania.
Pozwala to na ograniczenie aktywności botów, ponieważ sam koszt obliczeniowy wielu zadań logicznych jest zbyt wysoki dla twórcy bota[22].
Rozwiązanie to wydaje się optymalne, gdyż nie przeszkadza użytkownikowi, a jednocześnie odstrasza boty. Gdyby jednak cały świat nagle zaczął używać mechanizmów PoW, globalne zużycie energii elektrycznej widocznie by wzrosło.
Cena za bezpieczeństwo przed botami nie powinna być okupiona szkodą ekologiczną. Dodatkowo system Cloudflare Turnstile, który także bazuje na PoW, osiąga wykrywalność botów na poziomie 33%, co pokazuje, że nawet ten system weryfikacji jest przepuszczalny[23].

Alternatywą odchodzącą od rozwiązań tekstowych, obrazkowych czy też testów PoW są systemy ZKP (ang. *zero-knowledge proofs*).
Oparte są one na weryfikacji biometrycznej; maszyna użytkownika na bazie jego biometrii generuje dowód matematyczny.
Problemem wielu rozwiązań wykorzystujących biometrię jest zagrożenie wyciekiem danych, jednak w systemach ZKP wszystkie dane wrażliwe pozostają na maszynie użytkownika, nigdy jej nie opuszczając[24].
Podczas samej weryfikacji używany jest właśnie ten wygenerowany dowód matematyczny.
Gdy weryfikator otrzyma taki dowód, musi go zaakceptować, jednocześnie nie wiedząc, kim dokładnie jest użytkownik, co zapewnia prywatność.
Jedynym mankamentem ZKP jest sam problem wdrożenia na urządzeniach starszej generacji, które mogą być mniej wydajne i nie sprostać narzutowi obliczeniowemu.

[20]Źródło: arxiv.org (dostęp 9.03.2026 r.).
[21]Źródło: mdpi.com (dostęp 9.03.2026 r.).
[22]Źródło: roundtable.ai (dostęp 9.03.2026 r.).
[23]Źródło: roundtable.ai (dostęp 9.03.2026 r.).
[24]Źródło: researchgate.net (dostęp 9.03.2026 r.).

5 Dlaczego Nie Warto Tworzyć Własnych CAPTCHA

Jak zostało wspomniane poprzednio, weryfikacje tekstowe i grafiki CAPTCHA to w obecnych czasach przeżytek, a wprowadzanie podobnych technologii na własną rękę zwiększa jedynie złudne poczucie bezpieczeństwa i kontroli.
Jeśli jednak ktoś postanowił zbudować taki system, jest kilka problemów, na które natrafi.
Pierwszym problemem jest trudność samych testów weryfikacji – zakładając, że obecne boty nie potrafią ich tak łatwo rozwiązywać – powinny one być **łatwe** dla człowieka i trudne dla maszyny[25].
Dla przykładu używanie znaków podobnych do siebie – *patrz rysunek 5* – może wydać się sensowne, ponieważ bot będzie miał ogromne problemy z ich rozpoznaniem, ale w tym samym czasie przeciętny użytkownik będzie w podobnej sytuacji – nie mówiąc już o osobach z niepełnosprawnościami.



(a) CAPTCHA `\&1O]` (b) CAPTCHA `(oilO\<`

Rysunek 5: Przykładowe autorskie kody CAPTCHA, które zawierają podobne znaki[26].

Innym problemem jest przechowywanie obrazków lub tekstów CAPTCHA, jako że magazynowanie ich na maszynie użytkownika mija się z celem – adekwatna baza danych jest wymagana.
Sama liczba zapytań nie będzie rzeczą bagatelną, ale musimy także pamiętać o wielkości każdego obrazka.
Jeśli system ma być *„bezpieczny”*, potrzebujemy dużej liczby testów weryfikacji – tak, by bot nie nauczył się wszystkich kombinacji.
Kolejnym mankamentem jest problem legislacyjny, jako że coraz więcej państw i rządów tworzy wymogi dla twórców stron internetowych, tak by te były inkluzywne dla osób z niepełnosprawnościami.
Powyższe powinno się także potraktować z perspektywy etycznej, gdzie, jako ofertodawca takiej usługi, nie powinniśmy wykluczać żadnych użytkowników tylko dlatego, że źle zaprojektowaliśmy nasz system weryfikacji.

W przypadku rozwijania usług weryfikacji podobnych do ReCAPTCHA w wersji trzeciej, FriendlyCAPTCHA czy rozwiązań *zero knowledge*, sytuacja jest tym bardziej bezsensowna.
Bo choć wymienione usługi – z wyjątkiem ReCAPTCHA – oferują prywatność i ochronę, to samo ich zaprojektowanie i wdrożenie jest nieosiągalne bez ogromnych nakładów finansowych i czasowych.

[25]Jest to podstawowa definicja testów weryfikacyjnych CAPTCHA, stworzona przez Yahoo! i CMU.
[26]Tego typu znaki nie występują w końcowej implementacji, zostały jednak włączone dla powyższych celów pokazowych.

6 Implementacja

Wpierw funkcja *generate_strings* generuje ciągi znaków – od 5 do 8 znaków.
Te ciągi następnie są dodawane do zmiennej słownika – str_combinations – a następnie trafiają do funkcji, która eksportuje dane z powyższej zmiennej do pliku tekstowego.
Choć cały proces można by zrobić w jednej funkcji, rozdzielenie stworzenia ciągu i przesłania go do pliku jest czystsze.

Listing 1: Generator ciągów znaków w Pythonie

```
        def generate_strings(IMG_NUMBER: int):
            str_combinations = dict()

            for i in range(IMG_NUMBER):
                n = random.randrange(5, 9)
                comb = ""
                for _ in range(n):
                    while True:
                        character = random.randint(35, 123)
                        if (chr(character) not in
                            ("-", "_", "1", "~", "`", "!", "I", "l",
                            "0", "O", '"', "'", "}", "{", "(", ")",
                            ";", ":", "/", "\\", ",", ".", "[", "]", "i"
                            ,'t', "&", "=", "+", ">", "<", "j", 'o')
                            and chr(character) not in comb):
                                comb += chr(character)

                    str_combinations[str(i)] = comb
            strings_to_file(str_combinations)

```

Następnie funkcja *create_images* gotowe ciągi znaków przygotowuje do samego tworzenia obrazów. Nowe zmienne *reverse_color* i *color* odpowiadają zarówno za kolor tła, jak i za kolor samego tekstu – czarny lub biały – podobnie losowo wybierany.
Każdy znak w ciągu znaków ma przydzieloną losową liczbę spacji z przedziału, tak by znaki były cięższe do odczytania.

Listing 2: Ciągi znaków przygotowywane do generacji obrazów.

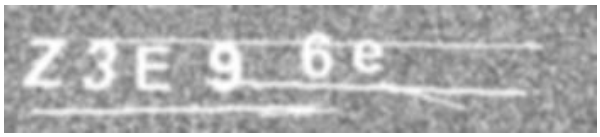
```
            str_combinations = dict()
            reverse_colors = False
            color = ['white', 'black']

            with open(file_location, 'r') as f:
                for i in range(IMG_NUMBER):
                    line = f.readline().strip("\n").split(";")
                    str_combinations[line[0]] = line[1]

            for i in range(IMG_NUMBER):
                reverse_colors = bool(random.randint(0, 2))
                color = sorted(color, reverse=reverse_colors)

                code = list(str_combinations[f"{i}"])
                text = "".join((code[i] + (random.randint(0, 3) * " "))

```

Skrypt działa na bibliotece *PIL*, która pozwala na rysowanie i generowanie obrazów. W zmiennych zostaje ustawiona szerokość i wysokość obrazu – wyrażona w pikselach. Następnie obraz zostaje konfigurowany, matryca 'L' – dla skali szarości – zmienne wielkości obrazu i kolor, losowany w poprzednim fragmencie.
Kolejno rozkład znaków zostaje zmieniony, ilość spacji pomiędzy znakami jest losowana. Dodatkowo rodzaj czcionki i wielkość czcionki są także losowane z przedziału.
Wszystkie te powyższe operacje tworzą unikatowe rozłożenie znaków już na samym obrazie. W następnym kroku program rysuje losową ilość linii, które zaczynają się i kończą w losowych miejscach i mają losowaną szerokość – same linie są w kolorze tekstu[27].
Następnie generowane są kropki w losowych miejscach obrazu; są one w kolorze tekstu. Tworzony zostaje efekt szumu – jego moc jest losowana – który następnie jest nakładany na sam obraz.
Przed zapisaniem obrazu i końcowym hashowaniem ciągów znaków na obraz nakładany jest filtr – rozmycie Gaussa.

Listing 3: Ciągi znaków dodawane na obrazek, który następnie zostaje zniekształcany.

```
            im_width = 350
            im_height = 75
            img = Image.new("L", (im_width, im_height), color[0])
            draw = ImageDraw.Draw(img)

            current_x = 10
            for j in range(len(text)):
                ft = ImageFont.truetype(["arialbd.ttf", "arialbi.ttf"]
                [(random.randint(0, 1))], random.randint(35, 40))

                draw.text((current_x, random.randint(5, 18)), text[j],
                fill=color[1], font=ft)

                current_x += draw.textlength(text[j], ft)

            for _ in range(random.randint(3, 7)):
                draw.line((random.uniform(0, im_width),
                random.uniform(0,im_height),random.uniform(0, im_width),
                random.uniform(0, im_height)), fill=color[1],
                width=random.randrange(3, 5))

            for _ in range(randint(5000, 7000)):
                draw.point(((random.randint(0, im_width)),
                (random.randint(0, im_height))), fill=color[1])

            noise = Image.effect_noise((350, 75),
            random.randrange(30,45))

            img = ImageChops.add(img, noise)
            img = img.filter(ImageFilter.GaussianBlur
            (radius=random.uniform(1, 1.5)))

            img.save(f"{img_location}\\img_{i}.jpg", format="JPEG",
            optimized=True)

            encrypt_codes(str_combinations)

```

[27]W wersji pilotażowej linie są w kolorze tła; był to błąd z mojej strony.



(a) CAPTCHA czarno-biała: '*Z3E96e*'.

(b) CAPTCHA biało-czarna: '*YgM49K*'.

Rysunek 6: Przykłady wygenerowanych obrazów CAPTCHA, widać różną odległość pomiędzy znakami, różna wysokość, zamazanie, niewyraźne tło i losowe linie.

W kolejnym kroku ciągi znaków – ze zmiennej słownikowej – zostają zahashowane, tak by nie były zrozumiałe dla człowieka postronnego.
Od tego momentu będą tylko wymagane jako porównanie z odpowiedziami użytkownika na dany test weryfikacji – jeśli odpowiedź użytkownika, która także jest zahashowana, odpowiada hashowi danego testu weryfikacyjnego, użytkownik jest uznany za człowieka, co pozwala mu się zalogować do serwisu.

Listing 4: Ciągi znaków zostają zahashowane.

```
        def encrypt_codes(str_combinations: dict):

            global BASE_DIR
            file_location = BASE_DIR

            try:
                file_location += "\\web_page\\static"
            except FileNotFoundError or PermissionError:
                file_location += "\\web_page"

            for i, j in enumerate(str_combinations.values()):

                str_combinations[f"{i}"] = hashlib.sha512(j.strip('\n').
                strip(" ").encode()).hexdigest()

            with open(f"{file_location}\\captcha_codes.txt", 'w') as f:
                for i, j in enumerate(str_combinations.values()):
                    f.write(f"{i};{j}\n")

```

Po wszystkich operacjach jesteśmy gotowi do włączenia programu.
Same funkcje zarówno tworzenia ciągów znaków, jak i samych obrazów, są wywoływane w przypadku braku obrazów lub gdy warunek bool jest równy *True*.

Listing 5: Plik *main.py*, miejsce startu progarmu.

```
            if __name__ == '__main__':
                if bool(random.randint(0, 1)) or
                (not os.listdir(f"{BASE_DIR}\\static\\captcha_images")):

                generate_strings(IMAGE_NUMBER)
                create_images(IMAGE_NUMBER)

                app.run(debug=False)

```

Funkcja ekstraktuje z pliku już zahashowane kombinacje znaków, które następnie będą służyć do porównywania z odpowiedziami użytkownika.

Listing 6: Fukcja *get_hashes*.

```
def get_hashes():
    with open(f"{BASE_DIR}\\static\\captcha_codes.txt", 'r')
    as f:

        for i in f:
            line = list(i.split(";"))
            str_combination[line[0]] = line[1].strip("\n")
            .strip(" ")

```

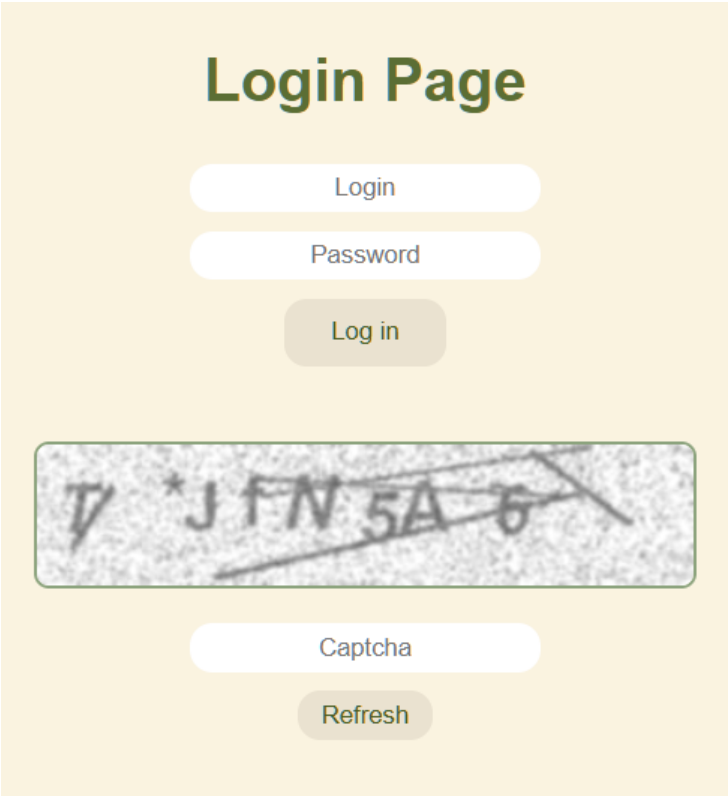
Po początku pliku *main.py* i ewentualnym wywołaniu funkcji tworzenia ciągów znaków i obrazów, przechodzimy do części interaktywnej.
Tutaj zaczynając od funkcji index, która wywołuje funkcję *get_hashes*, a następnie renderuje stronę główną – stronę logowania – *index.html*.
Użytkownik przy użyciu przeglądarki internetowej i wpisaniu prawidłowego adresu zostanie właśnie tam przekierowany.
Gdy użytkownik poprawnie wprowadzi dane – jakiekolwiek imię, jakiekolwiek hasło dłuższe niż 7 znaków i właściwy kod CAPTCHA – zostaje przekierowany na stronę *user_page.html*.
Dane użytkownika są przesyłane do skryptu – po tym, jak kliknie przycisk *Log in* – gdzie odpowiedź na test weryfikacji człowieczeństwa jest hashowana, tak by można sprawdzić zbieżność obu kodów.
W przypadku złych danych użytkownik nie może się zalogować, musi po raz kolejny wpisać dane. Jeśli kod weryfikacji CAPTCHA jest zbyt trudny, użytkownik może go zmienić za pomocą przycisku *Refresh*.

Listing 7: Flask łączy skrypty Python z frontendem – HTML i CSS.

```
@app.route('/')
def index():
    r_number = str(random.randint(0, IMAGE_NUMBER - 1))
    static_path = os.path.join(BASE_DIR, "static",
    "captcha_images")
    get_hashes()

    return render_template("index.html", r_number=r_number)

@app.route('/login', methods=['POST'])
def login():
    action = request.form.get('action')

    if action == 'refresh':
        r_number = str(random.randint(0, IMAGE_NUMBER - 1))
        return render_template("index.html",
        r_number=r_number)

    u_code = request.form.get('u_captcha')
    captcha_number = request.form.get("r_number")
    user_login = request.form.get("u_login")
    logged_in = False
    msg = ''

```

```
                    captcha_code = hashlib.sha512(u_code.encode())
                    .hexdigest()

                    if not logged_in:
                        if user_login is not None and
                        len(request.form.get('u_password')) > 7 and
                        captcha_code == str_combination[captcha_number]:
                            logged_in = True
                        else:
                            logged_in = False
                            msg = 'failure'

                    if logged_in:
                        return render_template("user_page.html",
                        user_login=user_login)
                    else:
                        r_number = str(random.randint(0, IMAGE_NUMBER - 1))
                        return render_template("index.html",
                        r_number=r_number, msg=msg)

```

Poniższy rysunek przedstawia stronę logowania – wraz z polami na login, hasło i kod weryfikacji CAPTCHA.
Sama struktura graficzna stron *index.html* i *user_page.html* została stworzona w językach HTML i CSS.



Rysunek 7: Strona logowania, po wpisaniu loginu, hasła i poprawnego kodu CAPTCHA, użytkownik może się zalogować.

7 Badanie AI

Badanie jest podzielone na dwie części – część pilotażową i część właściwą (wersję 2) – w obu częściach została przetestowana zdolność modeli AI do rozwiązywania wygenerowanych testów weryfikacji człowieczeństwa CAPTCHA, uwzględniono także rozwiązania człowieka[28].
Jednakże w badaniu pilotażowym prompt AI nie był homogeniczny, użyte zapytania były niezoptymalizowane, tak że każdy model mógł dostać inny prompt – nie było to kontrolowane.

Listing 8: Prompt podczas badania pilotażowego – inne prompty także mogły występować.

```
Cześ przygotowuję pracę o CAPTCHA, rozwiąż te captcha, i podaj mi
wyniki w takiej formie:
agaga
fafa
gagaga
fafpg
każda odpowiedź powinna być bez spacji.
odpowiedzi muszą być po kolei - np. odpowiedzi do img_0.jpg,
img_1.jpg, img_2.jpg itd.
```

Listing 9: Prompt podczas badania pilotażowego – inne prompty także mogły występować.

```
Cześć, rozwiąż dla mnie te obrazki - praca uniwersytecka.
obrazki zrób to w tej formie;
gsgsdfggs
bdgagg
ghgdfsg itd
w odpowiedzi daj mi tylko odpowiedzi jak powyżej.
w odpowiedzi usuń spacje
```

Kolejnym mankamentem w próbie pilotażowej były losowe linie, które generują się podczas tworzenia nowego obrazu. Ich ilość, grubość, punkt początku i końca są losowane z przedziałów, jednak były one w kolorze tła, a nie tekstu – *przypis 27* – co, choć dawało efekt, ostatecznie było pomyłką w implementacji.
Oba powyższe problemy zostały rozwiązane w wersji 2, gdzie prompt został zunifikowany, a problem linii rozwiązany.

(a) CAPTCHA z wersji pilotażowej

(b) CAPTCHA z wersji 2

Rysunek 8: Choć linie działają w wersji pilotażowej – 'przecinają' litery – to w wersji 2 widocznie funkcjonują jako dodatkowe utrudnienie.

[28]W badaniu pilotażowym jest 'dwóch' ludzi, jednak w rzeczywistości jest to jedna i ta sama osoba. Rozwiązania drugiej próby zostały podjęte w kolejnym dniu po pierwszej próbie – druga próba nie była po kolei, obrazy były sortowane po wielkości, nie po nazwie, jak w pierwszej próbie.

Listing 10: Prompt podczas badania w wersji 2 – jest on taki sam dla każdego modelu.

```
W ramach uniwersyteckiego projektu badawczego dotyczacego systemów
OCR i czytelnosci zniekształconego tekstu, prosze o dokonanie
dokładnej transkrypcji znaków z zalaczonych obrazów.

Obowiazują nastepujace rygorystyczne zasady formatowania:

-Zignoruj wszelkie tła i zakłocenia wizualne.
-Przepisz znaki z każdego obrazka, nie używajac wewnatrz nich ż
adnych spacji.
-Odpowiedź wygeneruj w formie prostej listy - kazdy ciąg znaków w
nowej linii.
-Twoja odpowiedź musi skladać sie w 100% z samych wynikPw. Nie
dolaczaj żadnego tekstu pobocznego ani formatowania markdown.
```

W przypadku modeli Gemini i modelu Grok prompty były używane w oknie czatu. Tam, po przekazaniu zapytania i dołączeniu obrazków, model AI generował swoje odpowiedzi. W przypadku *Dash.capMonster* wyniki były spisywane ręcznie do pliku tekstowego[29]. Pozyskane dane były obrabiane i porównywane w środowisku Jupyter Notebook – stamtąd pochodzą wykresy i tabele z danymi.

W obu próbach użyto próbki stu obrazów CAPTCHA, wygenerowanych za pomocą autorskiej implementacji.
W obu próbach użyto modeli Gemini 3 (tryb Pro), Gemini 3 (tryb Fast), Grok 4 (tryb Expert) i dash.capMonster (demo).
Czaty podczas części właściwej w przypadku Gemini 3 w trybie pro i fast były czatami tymczasowymi – nie jestem pewien, czy to wpływa na same wyniki – czaty Groka nie były czatami tymczasowymi.
W próbach pilotażowych dla Gemini używałem zarówno czatów tymczasowych, jak i standardowych.
Zarówno tryb pro (Gemini), jak i tryb expert (Grok) spędzają więcej czasu nad odpowiedzią niż tryb fast (Gemini).
W całym badaniu rozwiązania były rozpatrywane z perspektywy pięciu kategorii:

Full (Pełny) - gdy rozwiązanie w pełni pokrowało się z prawidłową odpowiedzą

Full Insensetive (Pełny niewrażliwy) - gdy rozwiązania pokrywały się w pełni, ale wielkość liter była inna – odpowiedz AI zawiera małe 'x', gdy odpowiedź wzorcowa zawiera duże 'X'.

Partial (Częściowy)- gdy roziwązanie pokrywało się z prawidłową odpowiedzią przynajmniej w 75%.

Partial Insensetive (Częściowy niewrażliwy)- gdy roziwązanie pokrywało się z prawidłową odpowiedzią przynajmniej w 75%, ale wielkość liter była inna.

Overall (Ogólny)- śrdenia arytmetyczna – zaokrąglona – z czterech powyższych (Full, Full Insensetive, Partial i Partial Insensetive).

[29]Usługa ta była w wersji demo, co nie pozwalało na kopiowanie - usługa generowała nowe kody CAPTCHA, które trzeba było odczytać i przepisać.

W tabeli przykładowych rozwiązań (tabela 1) z próby pilotażowej widzimy, że model Gemini 3 (tryb Fast) *halucynował*, część odpowiedzi to losowy ciąg znaków – choć dwa rozwiązania są w pełni prawidłowe.
Sam model Gemini - w trybie Pro i Fast - używał znaków alfabetu greckiego, gdy zarówno Grok, jak i Dash (dash.capmonster) z nich nie korzystali.

Tabela 1: Wyniki modeli AI, wersja pilotażowa

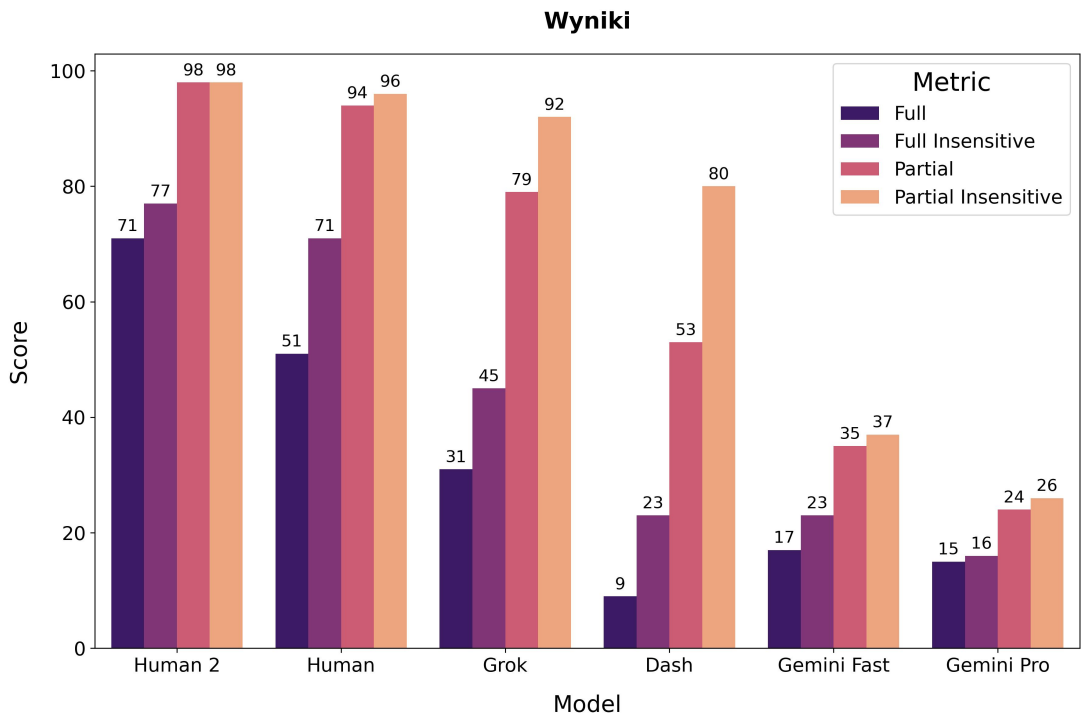
Correct	**Human**	**Human 2**	**Gemini Pro**	**Gemini Fast**	**Dash**	**Grok**
NZsy5	NZSy5	NZsy5	N<syE	accdbd	N2sy5	NZsy5
HE#rJY	HE#rJY	HE#rJY	HE#rJY	abccdc	HERJY	HE#rJY
?M^Ky	?M^Ky	?M^Ky	?M^Ky	eabdbc	MxKy	?M^Ky
nFPzT$y	nfPzT$y	nFPzT$y	rFPzT$y	acaecc	nFp2TSy	r+PZT$y
GrsDZVay	GrsDzVay	GrsDZVay	GrsDZVay	bdcaab	GrsDZVay	GrsDZVay
efVx?aEJ	efVx?aEJ	efVx?aEJ	efVx?aΞj	efVxraΞj	efvxra5j	efVx?aEj
ZVuQq	ZVuQq	ZVuQq	ZvuΩq	ZvuΩq	ZVuoq	ZvuQa
F3E#yMcY	F3e#yMcY	F3E#yMcY	F3Ξ#yMcY	F3Ξ#yM⊂Y	F3EMcY	F3=#yMcY
9zmcS	9zmcS	9zmcS	9zmcS	9zmcs	9zmcs	9zmcS
CvLZ?y	CvLZ?y	CvLZ?y	CvLZ?y	CvLZ?y	cvLzy	CVLZy

Końcowe wyniki pokazują, że człowiek – zarówno z próby pierwszej, jak i drugiej – dalej ma najwyższe wyniki w każdej kategorii. Należy jednak zaznaczyć, że wynik *Full* człowieka, plasujący się na poziomie 51% i 71%, jest zdecydowanie za niski jak na próbę identyfikacji człowieczeństwa – wynik powinien oscylować w okolicach 90%.
Powyższe może wskazywać na źle zaprojektowany test CAPTCHA, jest on zbyt trudny dla człowieka, co pokazują wyniki badania pilotażowego.
Sam model Grok osiągnął najwyższe wyniki ze wszystkich modeli AI, w kategorii *partial insensitive* plasuje się tuż za człowiekiem.
Kolejno Dash najgorzej wypada, jeśli chodzi o kompletnie poprawne odpowiedzi, jednak po Groku ma najwyższe wyniki w *partial* i *partial insensitive*.
Najciekawsza sytuacja występuje u modelu Gemini, gdzie to halucynujący tryb fast jest lepszy od trybu pro pod każdym względem – tryb pro poświęca parokrotnie więcej czasu na przesłanie odpowiedzi.

Tabela 2: Podsumowanie wyników rozwiązań - sortowanie wg. *Overall*

Model	**Full**	**Full Insens.**	**Partial**	**Partial Insens.**	**Overall** [%]
Human 2	71	77	98	98	86
Human	51	71	94	96	78
Grok	31	45	79	92	62
Dash	9	23	53	80	41
Gemini Fast	17	23	35	37	28
Gemini Pro	15	16	24	26	20

W próbie pilotażowej zdolność modeli do rozwiązywania testów weryfikacji człowieczeństwa jest wysoko rozbieżna.
Jedynie Grok 4 jest wystarczający – 31 poprawnych odpowiedzi ze 100 – w porównaniu do innych modeli.
Choć Dash ma najmniej odpowiedzi w pełni poprawnych, może to wynikać ze złego przepisania samych odpowiedzi lub nieumiejętności odczytania przez model znaków innych niż alfanumeryczne.



Rysunek 9: Wyniki pilotażowe w formie graficznej - wykres słupkowy

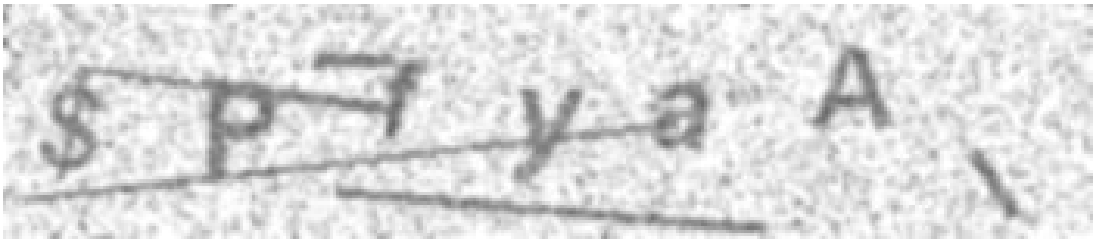
Wersja 2

Przechodząc do wersji 2 samego eksperymentu, widzimy kompletną zmianę w przypadku modelu Gemini w trybie Pro.
Z dziesięciu przykładowych wyników udało mu się rozwiązać aż 7 w pełni poprawnie, w próbie pilotażowej jego wynik wynosił siedemnaście na sto prób (Full).
Jednak w tym samym czasie tryb Fast nie trafił żadnych prawidłowych odpowiedzi. Wszystkie jego odpowiedzi – w tym fragmencie – są halucynacją i nie pokrywają się z poprawnymi odpowiedziami.

Ponownie Dash rozpoznaje sporą część znaków, jednak ani razu nie użył innych znaków niż alfanumeryczne – w dziesiątym teście (V8@%Y), choć poprawnie wskazuje znaki alfabetyczne (V i y), jak i cyfrę 8, to kompletnie pogubił się przy '@' i '%', używając '0' i '7' zamiast nich.

Tabela 3: Wyniki modeli AI, wersja 2

Correct	**Human**	**Gemini Pro**	**Gemini Fast**	**Dash**	**Grok**
^RqyYP	^RqyYP	^RgYP	7JRYX	R9yYp	^R9YP
HwyWU#M	HWyWU#M	HwyWU#M	EHHW9	HWyWUTM	HwywU#M
4%HTy	4%HTy	4%HTy	SSTB8	46HTy	4%HTY
zpuKnW#	zpuKnW#	zpuKnW#	79X8S	zpuknw4	ZpurKnW#
GLYQR	GLYQR	GLYQR	PFD59	GLYQR	GLYQR
mkB^9	mkB^g	mkBAg	8G8X6	mkB4S	mKB^G
rgzsKU	rgzsKU	rgzsKU	WPK8P	r9zsKU7	rQZSKU+
Ufyzupv	Utyzupv	Utyzxpv	M4X9N	ULyzuLpv	UyzZuTpV-
P2Jzu3Sa	P2Jzu3Sa	P2Ju3Sa	7M9XN	pwJzu3Sa	P2JZU3Sa-
V8@%y	V8@%y	V8@%y	F66PN	V807y	V8@%Y

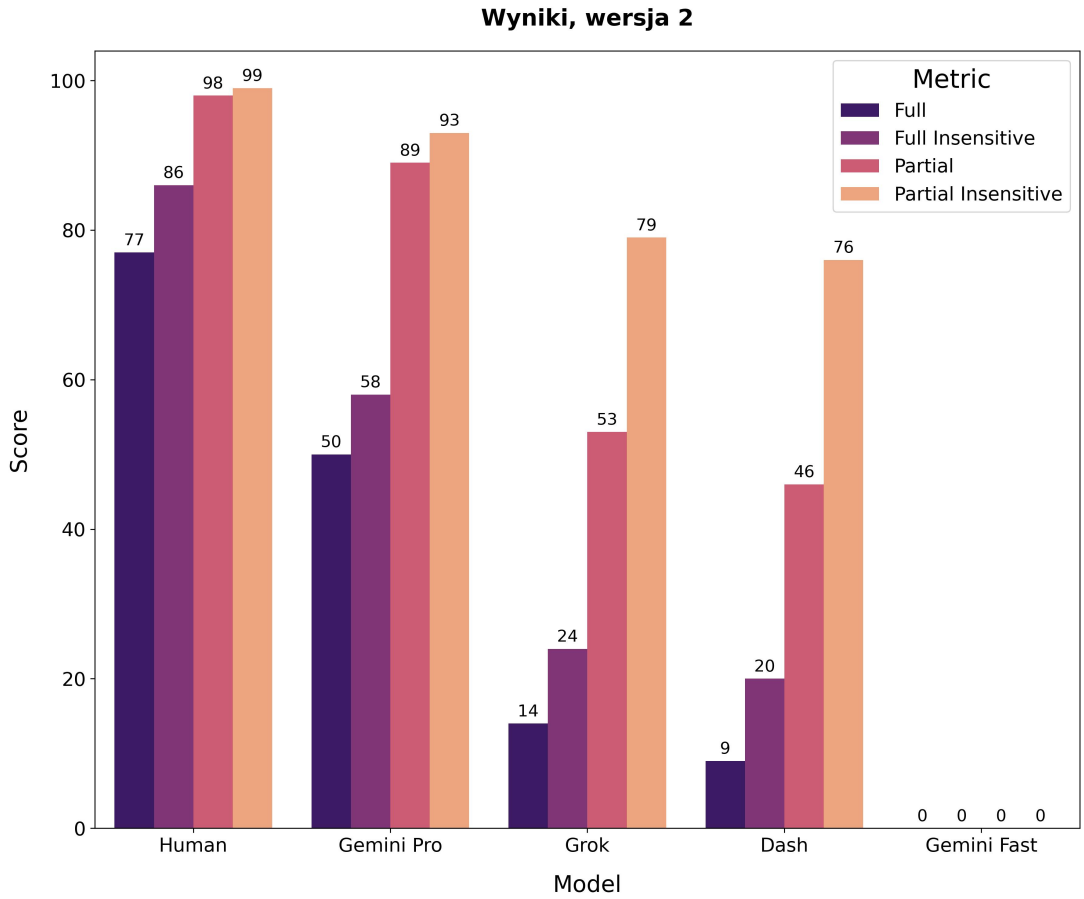


Rysunek 10: dash.capMonster odczytał ten rysunek jako '*SP7yaA*', znak *$* został odczytany jako *S*. Sam system najpewniej operuje tylko na znakach alfanumerycznych.

Patrząc na samo podsumowanie, powyższe zachowanie Gemini (fast) nie było tylko chwilową halucynacją – jak w próbie pilotażowej – lecz powtarzało się przez całą próbę.
Człowiek dalej pod każdym względem wygrywa z modelami, choć Gemini (pro) jest drugie w *partial* i *partial insensitive*.
Nie zmienia to faktu, że wynik *full* jest dalej za niski jak na test CAPTCHA.
Końcowo to jednak Gemini (pro), które poprzednio było najgorsze ze wszystkich modeli, tym razem wyprzedziło Groka i znajduje się tuż za człowiekiem.
Sama słabość modelu Grok i siła modelu Gemini (pro) musi wynikać albo z bardziej dokładnego promptu, albo z innego podejścia do CAPTCHA z dodatkowymi liniami.

Tabela 4: Podsumowanie wyników rozwiązań w wersji 2 - sortowanie wg. *Overall*

Model	**Full**	**Full Insens.**	**Partial**	**Partial Insens.**	**Overall** [%]
Human	77	86	98	99	90
Gemini Pro	50	58	89	93	72
Grok	14	24	53	79	42
Dash	9	20	46	76	38
Gemini Fast	0	0	0	0	0

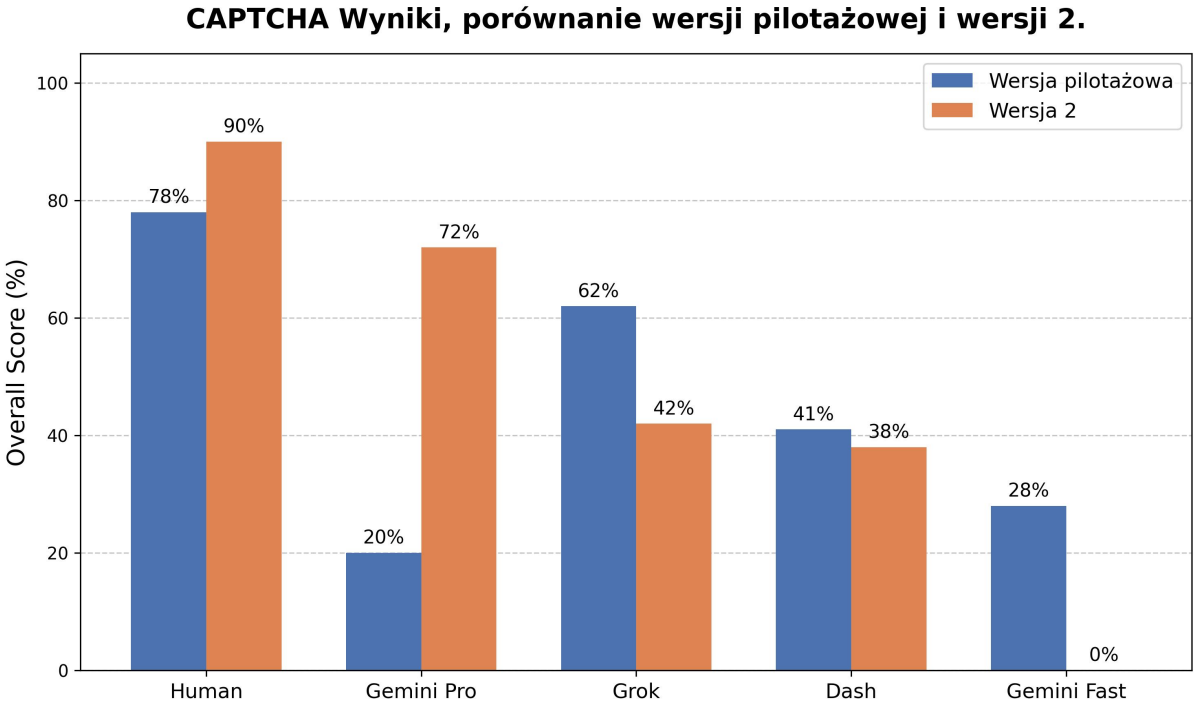


Rysunek 11: Wyniki wersji 2 w formie graficznej - wykres słupkowy

Przechodząc do porównania dwóch wersji – pilotażowej i właściwej – należy zaznaczyć, że nie można traktować ich na równi.
Rozbieżność promptów w wersji pilotażowej i standaryzacja promptów w wersji 2, a także różne rodzaje testów weryfikacji nie pozwalają na jednoznaczne stwierdzenie, czy wyniki powstały z powodu standaryzacji zapytania, czy zmiany implementacji generowania obrazów CAPTCHA.

Tabela 5: CAPTCHA wyniki porównawcze, wersja pilotażowa i wersja 2.

Model	**Wersja pilotażowa (%)**	**Wersja 2 (%)**
Human	78	90
Gemini Pro	20	72
Grok	62	42
Dash	41	38
Gemini Fast	28	0



Rysunek 12: Porównanie wersji pilotażowej i wersji 2 - wykres słupkowy

Jak widać na powyższych porównaniach, wersja 2 jest obiektywnie lepsza od wersji pilotażowej – w przypadku łatwości dla człowieka.
Wskazuje to na samą zmianę w implementacji, najwyraźniej ta wersja testu CAPTCHA jest łatwiejsza niż poprzednia.
Choć obu części badania nie można stawiać na równi, ta jest wyjątkiem.
Jako że sama konfiguracja promptów nie wpływa na wyniki człowieka – dla człowieka jedyną liczalną zmienną jest zmiana implementacji (zmiana koloru linii).

Widać ponad potrójną zmianę przy modelu Gemini (pro) – w części właściwej jego wynik poprawnych odpowiedzi wynosi 50, gdzie w części pilotażowej wyniósł 15.
Jednak kompletny spadek widzimy przy Gemini w trybie fast, trudno powiedzieć, czy halucynacje wynikły z samego promptu, zmiany rodzaju testu weryfikacji, czy też innego czynnika.
Sytuacja jest taka sama z trybem pro, nie da się stwierdzić, czy to sam prompt, czy zmiana testu dała tak dużą zmianę.
Sam Grok także widocznie spadł – ilość prawidłowych odpowiedzi spadła z 31 do 14, ilość odpowiedzi *full insensitive* spadła z 45 do 24.
Jednak jeśli chodzi o odpowiedzi *partial* i *partial insensitive*, to spadki nie są aż tak drastyczne.

Wyniki Dasha są jednak dość podobne, podobnie jak z człowiekiem, rodzaj promptów nie ma znaczenia, jedynymi zmiennymi są błędy przy samym przepisywaniu (pomylenie wielkości liter przez człowieka) i sama zmiana testów CAPTCHA.
Może to wskazywać na niski poziom trudności tekstowych testów CAPTCHA, choć wyniki Dasha są niskie, to wynikają one najpewniej z nieumiejętności odczytania znaków innych niż alfanumeryczne, które jednak występowały w tym systemie weryfikacji.

8 Wnioski i Refleksje

Choć sam temat CAPTCHA dla większości społeczeństwa może wydawać się sprawą błahą, to jej istnienie pozwala na istnienie internetu takim, jakim go znamy.
Zaczynając od samych początków teoretycznej gry imaginacji Alana Turinga aż po obecne rozwiązania ZKP, historia weryfikacji człowieczeństwa jest coraz dłuższa – a sama ewolucja tych systemów dalej jest sprawą otwartą.

Z perspektywy przeprowadzonych badań na modelach sztucznej inteligencji, które nie są wyspecjalizowane, wytrenowane do rozpoznawania testów CAPTCHA, widzimy, że nawet jeśli ich wyniki nie są przytłaczające, to stanowią o niebezpieczeństwie, jakie niesie używanie weryfikacji w formie tekstowej.
Nawet jeśli Gemini w trybie pro ma 50% szans na prawidłową odpowiedź, ta liczba jest i tak zbyt wysoka.
Wystarczy, by model po raz kolejny podszedł do testu weryfikacji, by przejść go pomyślnie.
Powyższe potwierdza stwierdzenie firmy Google sprzed lat, gdzie ich własna sztuczna inteligencja – wytrenowana na CAPTCHA tekstowych – z przytłaczającą poprawnością potrafiła je rozwiązywać.
Jednak tamten model był najpewniej modelem prywatnym, niekomercyjnym, jakim dziś jest Gemini.
Choć weryfikacje tekstowe spotyka się coraz rzadziej, to tak naprawdę nie są już testy weryfikacji, które są łatwe dla człowieka, a trudne dla maszyny – jeśli już, to są łatwe dla człowieka i nietrudne dla maszyny.
A tu jest mowa jedynie o popularnych, komercyjnych modelach AI; dla obecnych botów weryfikacje tekstowe są najpewniej niczym, gdy dla człowieka, a zwłaszcza takiego z niepełnosprawnością, te weryfikacje rodzą problemy i trudności, nie dając nic w zamian.
Testy graficzne CAPTCHA także straciły swój sens wraz z YOLO8, które jest w stanie nieomylnie odpowiedzieć na test ReCAPTCHA 2.
Test obrazkowy też może być stosowany – w zależności od ustawień – w naruszającym prywatność CAPTCHA 3, gdy ten wykryje podejrzane zachowanie.
Ten sam problem występuje z popularnym hCAPTCHA, bo co po prywatności, jeśli usługa nie daje protekcji.
A nawet jeśli bot nie może rozwiązać testu, to kontaktuje się z farmą CAPTCHA, gdzie człowiek robi to za niego.
FriendlyCAPTCHA i CloudFlare Turnstile, bazujące na PoW, rozwiązują powyższe problemy – lecz samo użycie ich na skalę światową przyniosłoby globalnie większy koszt energetyczny.

Jedynym sensownym rozwiązaniem jest ZKP; trudno jest je podrobić, nie stanowią problemu dla prywatności użytkownika, nie stanowią utrudnień dla osób z niepełnosprawnościami i nie powinny stanowić.
Na obecną chwilę jest to rozwiązanie trudne do wprowadzenia na starsze maszyny, jednak sytuacja może się zmienić w przyszłości. Wraz z urządzeniami nowszej generacji i powolnym wyparciem starszych modeli, system weryfikacji biometrycznej ZKP będzie mógł się stać szeroko używanym standardem.
Co, mając nadzieję, uczyni internet bardziej przyjaznym i bezpiecznym miejscem.

Glosariusz

AI-hard problem - jest to problem, który wymaga zaawansowanej sztucznej inteligencji, aby go rozwiązać; historycznie rozpoznawanie obrazów przez komputer zaliczało się do tej kategorii.

Bot - jest to program komputerowy udający użytkownika.
Może być użyty, aby wejść na określoną stronę internetową i uzyskać z niej informacje; może też służyć jako narzędzie ataku na stronę internetową.

CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) - jest to test weryfikujący bycie człowiekiem; powinien być on łatwy dla człowieka, ale trudny dla maszyny.

CSS (Cascading Style Sheets) - jest to język programowania, który we współpracy z HTML-em pozwala na graficzne upiększenie strony internetowej – nie jest on jednak wymagany.

Dowód pracy (PoW) - jest to rodzaj CAPTCHA polegający na zadaniu maszynie użytkownika matematycznej łamigłówki, której rozwiązanie zużywa moc obliczeniową. Rozwiązanie to ma odstraszać boty; ponieważ rozwiązywanie tego typu zadań jest nieopłacalne, staje się to nieekonomiczne dla twórców botów.

Farma CAPTCHA - gdy bot, próbując dostać się na stronę, napotka test CAPTCHA, który jest zbyt ciężki do rozwiązania, wysyła go do farmy CAPTCHA.
Tam żywi ludzie rozwiązują ten test za niego – są to najczęściej osoby z krajów rozwijających się.

HTML (Hypertext Markup Language) - jest to język programowania, który pozwala na stworzenie wizualnej części strony internetowej.

OCR (Optical Character Recognition) - jest to rozwiązanie technologiczne pozwalające na odczytywanie fizycznych znaków przez komputer.

Prompt - jest to najczęściej zapytanie tekstowe, które stanowi swoistą komendę dla modelu sztucznej inteligencji; sam prompt ma wpływ na jakość odpowiedzi AI.

Python - jest to język programowania obecnie często używany przy trenowaniu modeli sztucznej inteligencji; oprócz tego często wykorzystuje się go przy pracy z dużymi zbiorami danych.

SHA (Secure Hash Algorithms) - jest to rodzina deterministycznych algorytmów haszowania, które pozwalają na bezpieczniejsze przechowywanie wrażliwych danych.

String - ciąg znaków; jest to rodzaj zmiennej w wielu językach programowania, która przechowuje znaki graficzne.
W języku Python typ String wygląda następująco: ciąg_znaków = "Hello World!".

Sztuczna inteligencja (AI) - są to programy komputerowe, które symulują inteligentne zachowania.
Poprzez proces *głębokiego uczenia się* modele mogą stawać się coraz lepsze.

Bibliografia

1. NotebookLM. *Note-taking app by Google, leveraging AI technology.* notebooklm.google.com (link do zeszytu)
2. Drawing captcha *Mohammad Shirali-Shahreza – Sharif University of Technology.* https://www.researchgate.net/figure/An-Altavista-CAPTCHA-word-2_fig1_224651246
3. Breaking a Visual CAPTCHA *Greg Mori – UC Berkeley Computer Vision Group, Simon Fraser University; Jitendra Malik UC Berkeley Computer Vision Group.* https://www.cs.sfu.ca/~mori/research/gimpy/
4. CAPTCHA vs. reCAPTCHA: What's the difference? *Paige Tester – DataDome.co.* https://datadome.co/guides/captcha/hcaptcha-vs-recaptcha/
5. Captcha Test for Fun: More Than Just Security *GeeTest Support Team.* https://www.geetest.com/en/article/captcha-test-for-fun
6. Xu, Z., Yan, Q., Yu, F. R., & Leung, V. C. M. (2023). *A Survey of Adversarial CAPTCHAs on its History, Classification and Generation.* arXiv:2311.13233.
7. Igbekele, E. O., Adebiyi, A. A., Ibikunle, F. A., Adebiyi, M. O., & Olugbara, O. O. (2021). *Research trends on CAPTCHA: A systematic literature.* International Journal of Electrical and Computer Engineering (IJECE), 11(5), s. 4300–4312.
8. Avanesi, V., & Teurlings, J. (2022). *"I'm Not a Robot," or am I?: Micro-Labor and the Immanent Subsumption of the Social in the Human Computation of ReCAPTCHAs.* International Journal of Communication (IJoC), 16, s. 1441–1459.
9. Walia, J. S., & Odugoudar, A. (2024). *Vulnerability analysis of captcha using Deep learning.* arXiv:2302.09389v2.
10. Shojae Chaeikar, S., Mirzaei Asl, F., Yazdanpanah, S., Zamani, M., Manaf, A. A., & Khodadadi, T. (2023). *Secure CAPTCHA by Genetic Algorithm (GA) and Multi-Layer Perceptron (MLP).* Electronics, 12(19), 4084.
11. Gazi University. (2025). *Revisiting Text-Based CAPTCHAs: A Large-Scale Security and Usability Analysis Against CNN-Based Solvers.* Electronics, 14(22), 4403.
12. Algwil, A. (2023). *A SURVEY ON CAPTCHA: ORIGIN, APPLICATIONS AND CLASSIFICATION.* Journal of Basic Sciences (JBS), 36(1).
13. Plesner, A., Vontobel, T., & Wattenhofer, R. (2024). *Breaking reCAPTCHAv2.* In 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC).
14. Moreno, L., González, M., & Martínez, P. (2014). *CAPTCHA and accessibility. Is this the best we can do?.* Universidad Carlos III de Madrid (DRAFT VERSION WEBIST'2014).
15. Samson, F. (2025). *ZERO-KNOWLEDGE PROOFS FOR BIOMETRIC IDENTITY VERIFICATION.* ResearchGate.

16. Dayanand, Jeberson, W., & Jeberson, K. (2024). *Unveiling CAPTCHA Vulnerabilities: Breaking CAPTCHA Using Deep Learning Techniques and Design and Development of Robust CAPTCHA Technique*. SSRG International Journal of Electrical and Electronics Engineering, 11(6), s. 282–304.

17. Worcester Research and Publications. (2010). *CAPTCHA Accessibility Study of Online Forums*. University of Worcester.

18. MDPI Sensors. (2023). *New Cognitive Deep-Learning CAPTCHA*. Sensors, 23(4).

19. Artykuł naukowy. (2023). *Gotta CAPTCHA 'Em All: A Survey of Twenty years of the Human-or-Computer Dilemma*. Unpublished working draft.

20. THE Journal. (2025). *New Research Confirms AI Can Defeat Image-Based CAPTCHAs*.

21. Berton, R., Gaggi, O., & Kolasińska, A. (2022). *A study on Accessibility of Google ReCAPTCHA Systems*. In Proceedings of the 8th International ACM SIGACCESS Conference on Computers and Accessibility.

22. Yalamanchili, S. (2011). *CAPTCHA: DevaCAPTCHA framework*. arXiv preprints.

23. Google Search Central Blog. (2026). *Introducing reCAPTCHA v3: the new way to stop bots*.

24. SRR Publications. (2024). *Exploring the integration of machine learning techniques within CAPTCHA systems to dynamically adjust*. International Journal of Scholarly Research in Multidisciplinary Studies, 4(2), s. 1–7.

25. Ronis, J. (2023). *Don't Trust When You Can Verify: A Primer on Zero-Knowledge Proofs*. Wilson Center, Science and Technology Innovation Program.

26. ScrapingAPI.ai. (2025). *CAPTCHA Wars: Anti-Scraping Statistics and Bot vs Human Success Rates in 2025*.

27. W3C APA Working Group. (2021). *Making Content Usable for People with Cognitive and Learning Disabilities*. W3C Working Draft.

28. Hecks, E. (2025). *The Accessibility Problem With Authentication Methods Like CAPTCHA*. Designerly Magazine / Smashing Newsletter.

29. W3C WAI. (2026). *W3C Accessibility Guidelines (WCAG) 3.0 Working Draft*.

30. Friendly Captcha. (2026). *reCAPTCHA v3 Guide*.

31. Friendly Captcha. (2026). *reCAPTCHA v2 vs v3: Effective Bot Protection? (2026 Update)*.

32. Raport badawczy. (2023). *User Perception of CAPTCHAs: A Comparative Study between University and Internet Users*.

33. SoS-VO. (2025). *Publications | Science of Security Virtual Organization*.

34. Centre For Accessibility Australia. (2025). *W3C Web Accessibility Initiative: 2025 Year in Review*.